Kotlin Coroutines — Cheat Sheet

A dense personal study guide · Read this and you have the working knowledge

25 sections | Tables · Code · No fluff | 2026 Edition

About this sheet: a personal high-density study aid covering Kotlin Coroutines from the language model down to advanced Flow patterns and testing. Designed as a reference for someone who has read (or is reading) *Kotlin Coroutines: Deep Dive* by Marcin Moskała — not a substitute. Concepts are written entirely in my own words; examples are standard idiomatic patterns. Read top-to-bottom for the first pass; skim sections later for re-recall.

1. The mental model in 60 seconds

A *coroutine* is a suspendable computation: it can pause itself at well-defined points and later resume on the same or a different thread, without blocking the underlying thread while it waits.

- Threads are scheduled by the OS, cost ~1 MB stack each, and switch with kernel overhead. Coroutines are scheduled by the Kotlin runtime, cost ~hundreds of bytes (one `Continuation`), and "switch" by returning to a state machine.
- You can spawn millions of coroutines on a single CPU core; you cannot spawn millions of threads.
- The compiler converts `suspend` functions into state machines that take a `Continuation` parameter. Calling another suspend function may pause the current one until the callee resumes.
- "Async-looking" code reads top-to-bottom like blocking code, but doesn't block.

```
suspend fun loadUser(): User {
    val json = api.fetch()        // suspends here
    return json.toUser()           // resumes here when ready
}
```

Three things to internalize:

1. **Suspension is cooperative.** Only `suspend` functions can pause. Inside one, the framework decides where to resume.
2. **Structured concurrency.** Every coroutine has a parent scope; when the parent ends or fails, all children are cancelled.
3. **Cold by default for Flow, hot for Channels and StateFlow.** Cold sources do nothing until collected; hot sources run regardless of subscribers.

2. Suspension under the hood

The Kotlin compiler rewrites every `suspend` function into a Java/JVM method that takes an extra `Continuation<T>` parameter and returns either the result or the special marker `COROUTINE_SUSPENDED`. Suspension points become labels in a generated state machine.

```
// You write:
suspend fun work(): Int {
    val a = stepOne()       // suspension point #1
    val b = stepTwo(a)      // suspension point #2
    return a + b
}

// Compiler emits roughly (simplified):
fun work(cont: Continuation<Int>): Any? {
    val sm = (cont as? WorkStateMachine) ?: WorkStateMachine(cont)
    when (sm.label) {
        0 -> { sm.label = 1; val r = stepOne(sm); if (r == COROUTINE_SUSPENDED) return r; sm.a = r as Int }
        1 -> { sm.label = 2; val r = stepTwo(sm.a, sm); if (r == COROUTINE_SUSPENDED) return r; sm.b = r as Int }
        2 -> return sm.a + sm.b
    }
}
```

Implications:

- Local variables across suspension points are stored as fields on the Continuation/state-machine object.
- "Suspension" = the function returns `COROUTINE_SUSPENDED` to its caller, having stored everything; the awaited operation later calls `cont.resumeWith(...)`, which jumps to the next state.
- A coroutine is not pinned to a thread between suspensions — it can resume on a different thread (decided by the dispatcher).
- This is what's called **Continuation-Passing Style** transformation.

★ Internals — what's NOT magic (the things you assumed were magic)

The 60-second model in §1 and the state-machine walkthrough in §2 explain how `suspend` works. Here's the same demystification applied to *every other* piece of coroutines that people treat as magic. After this section, "advanced" stops feeling mysterious — every "magical" behavior turns out to be a small piece of plain Kotlin code.

1. `launch { }` "starts a coroutine" — full mechanics

> **Looks like magic:** calling `launch` somehow makes a block run in parallel.
> **Actually:** `launch` is a regular extension function on `CoroutineScope`. Step by step:

```
// Conceptual signature
fun CoroutineScope.launch(
    context: CoroutineContext = EmptyCoroutineContext,
    start: CoroutineStart = CoroutineStart.DEFAULT,
    block: suspend CoroutineScope.() -> Unit
): Job {
    val newCtx = newCoroutineContext(context)        // 1. merge parent ctx + supplied ctx
    val coroutine = StandaloneCoroutine(newCtx, ...)  // 2. build the coroutine instance (IS-A Job + Continuation)
    coroutine.start(start, coroutine, block)          // 3. ask dispatcher to schedule first resumption
    return coroutine                                  // 4. return the Job
}
```

- **(1)** `newCoroutineContext` combines parent context + supplied context. Crucially, it replaces the parent's Job with the new coroutine's Job — that's how the parent-child link forms.
- **(2)** A `StandaloneCoroutine` is one object that implements both `Job` (so cancellation works) and `Continuation` (so the suspending body can resume into it).

- **(3)** `start()` calls the dispatcher's `dispatch(continuation)`, which queues the first resumption. The block doesn't run synchronously — it runs whenever the dispatcher's worker picks it up.
- **(4)** The Job is returned so you can `cancel()`, `join()`, or `invokeOnCompletion { }`.

`CoroutineStart` modes worth knowing:

- **DEFAULT** — dispatch immediately (most common).
- **LAZY** — don't dispatch until `start()`, `join()`, or `await()` is called.
- **ATOMIC** — dispatch immediately and ignore early cancellation (block always runs at least to first suspension).
- **UNDISPATCHED** — run inline on caller thread until first suspension; then resume per dispatcher.

1b. `async { }` — what changes vs launch

> **Looks like magic:** async returns a "future-like" thing that can throw later.
> **Actually:** Same machinery as launch, but the returned object is a `Deferred<T>` (which IS-A `Job` with an extra "result slot"). The block's return value goes into that slot; `await()` reads it.

```
val d: Deferred<User> = scope.async { api.user() }
val u: User = d.await()      // suspends until result; throws if block threw
```

- `await()` uses `suspendCancellableCoroutine`: if the Deferred is already complete, returns the result immediately; otherwise registers a callback via `invokeOnCompletion` and suspends. When the block completes, that callback resumes the awaiter with the result (or rethrows the stored exception).
- **Exception behavior**: an exception in the block is stored in the Deferred. It's rethrown only when you call `await()`. *But* the failure also propagates up the Job tree — which means in a non-supervised scope, even an unawaited `async` exception cancels the parent scope. In a `supervisorScope`, the exception just sits in the Deferred until somebody awaits.
- **Don't** use `async {...}.await()` back-to-back — it's the same as a `suspend` call but with the overhead of building a Deferred. Use async only when you fork into parallel work.

2. Cancelling the parent kills all children

> **Looks like magic:** one `cancel()` propagates through every nested coroutine.
> **Actually:** `Job` keeps a list of children, populated when a child is created via `attachChild`. `cancel()` walks that list and calls `cancel()` on each. Each child does the same to its own children. It's just a tree traversal.

```
// Roughly what cancellation propagation looks like:
fun Job.cancelTree() {
    children.forEach { it.cancelTree() }    // recurse first
    state = Cancelling
    completion.invokeAll(CancellationException())
}
```

3. `coroutineScope { }` waits for everything inside

> **Looks like magic:** a function call somehow waits for arbitrarily many child coroutines.
> **Actually:** It creates a fresh `Job`, runs your block in a new scope using that Job as parent. Your block returns, and the new Job goes to *Completing* state. It only goes to *Completed* when all children finish. Then the outer suspend resumes. No magic — just `Job.join()` on the new scope's Job, internally.

4. `Dispatchers.Main` knows about Android's UI thread

Looks like magic: a generic `kotlinx.coroutines` dispatcher somehow knows about Android.
Actually: Java's `ServiceLoader` mechanism. The `kotlinx-coroutines-android` artifact ships a `META-INF/services/kotlinx.coroutines.internal.MainDispatcherFactory` file pointing at `AndroidDispatcherFactory`. On first access, that factory is loaded and produces a dispatcher that posts to the main `Looper`'s `Handler`. Pure JVM apps without that artifact get a "missing main dispatcher" error or a fallback dispatcher.

5. `CoroutineContext` lets you `+` together random elements

Looks like magic: `Dispatchers.IO + CoroutineName("x") + SupervisorJob() + handler` — how do unrelated types compose?
Actually: `CoroutineContext` is a persistent, immutable map keyed by `CoroutineContext.Element.Key`. Every element (`Job`, `CoroutineDispatcher`, `CoroutineName`, `CoroutineExceptionHandler`) declares its own Key and IS-A Element. `+` calls `plus()` which builds a combined context (essentially a linked list of elements). `context[Job]` walks the list looking for the element with that Key.

6. Cold `Flow` "doesn't run" until collected

Looks like magic: `val f = flow { println("hi") }` — nothing prints. Until `f.collect {}`. Then it prints. Repeat `collect` — prints again.
Actually: `flow { ... }` is a builder that returns an object with a single method `collect(collector)`. That method runs your lambda, passing it a collector to `emit` to. Until something calls `collect`, your lambda is just stored as a function reference. There's no scheduler, no queue, no lifecycle — just "nobody invoked the function yet." Coldness is not a feature; it's the absence of magic.

7. `emit()` and the collector body run synchronously, on the same coroutine

Looks like magic: producer "talks to" consumer; somehow values flow through.
Actually: `emit(x)` directly calls `collector.emit(x)`, which directly calls the lambda you passed to `collect { ... }`. No threads, no buffer, no queue — just nested function calls on the same call stack. That's why backpressure is automatic: the producer literally cannot run faster than the consumer because they're the same code path.

```
flow { emit(1); emit(2) }.collect { print(it) }
// Equivalent to (roughly):
val collector = FlowCollector { print(it) }
collector.emit(1)              // runs print(1)
collector.emit(2)              // runs print(2)
// All on the calling coroutine. No threads created.
```

8. `flowOn` switches the upstream dispatcher only

Looks like magic: how does it know where the boundary between "upstream" and "downstream" is, and switch threads only above it?
Actually: `flowOn` introduces a `Channel` between upstream and downstream. Upstream operators run on the new dispatcher and `send` to the channel; the collector runs on the original dispatcher and `receive`s from it. The channel *is* the boundary. That's why you'll see a small extra cost — there's a real channel hop.

9. `StateFlow` "drops equal values"

> **Looks like magic:** set `value = x` twice with same value, subscribers fire only once.
> **Actually:** The `value` setter does an internal `compareAndSet` (real CAS via `AtomicReference`). If the new value `equals` the current, no-op. Otherwise the new value is published and subscribers get notified. *This is why a data class with broken `equals` can cause unwanted recompositions.*

10. `SharedFlow` broadcasts to many collectors

> **Looks like magic:** one `emit` reaches every subscriber.
> **Actually:** SharedFlow internally maintains a list of "subscriber slots", each tracking that collector's read index in a circular buffer. `emit` walks the slots, advancing each subscriber's available index, and unparks any that were waiting. `replay` = the size of the circular buffer of "old values" shown to new subscribers when they join.

11. `callbackFlow` + `awaitClose` "blocks forever" until cancel

> **Looks like magic:** `awaitClose { }` just sits there suspended, then somehow runs cleanup at the right moment.
> **Actually:** `callbackFlow` is built on a `ProducerScope` that owns a `Channel`. `awaitClose` registers a callback on that channel's `invokeOnClose` hook and suspends the producer coroutine. When the collector cancels, the channel is closed; `invokeOnClose` fires; `awaitClose` resumes; your cleanup block runs; coroutine completes.

12. `yield()` "checks for cancellation"

> **Looks like magic:** a one-line call somehow notices cancellation in a tight loop.
> **Actually:** `yield()` calls `coroutineContext[Job]?.ensureActive()` first. If the Job is cancelled, that throws `CancellationException` immediately. If still active, it then dispatches the continuation back through the dispatcher (giving other coroutines a chance to run). `ensureActive()` is the cancellation check; the dispatcher hop is the "give others a turn" part.

13. `CoroutineExceptionHandler` "only fires for root coroutines"

> **Looks like magic:** sometimes my handler fires; sometimes the same exception silently kills siblings instead.
> **Actually:** When a coroutine throws, the exception walks up the parent Job chain. At each non-root Job, the exception *cancels the parent* (which then cancels all its children — cascading failure). Only when the exception reaches a root Job (a Job whose `parent` is null — the Job sitting at the top of your `CoroutineScope`) is the `CoroutineExceptionHandler` from that scope's context invoked. That's why handlers attached to nested launches are ignored.

14. `viewModelScope` "auto-cancels" when ViewModel is cleared

> **Looks like magic:** a property on ViewModel knows about ViewModel lifecycle.
> **Actually:** First access to `viewModelScope` creates `CoroutineScope(SupervisorJob() + Dispatchers.Main.immediate)` and stores it as a `Closeable` on the ViewModel via `setTagIfAbsent` / `addCloseable`. When `ViewModel.onCleared()` runs, the framework calls `close()` on every registered Closeable. The Closeable wrapper calls `scope.cancel()`. Same trick powers `lifecycleScope` (cancelled by Lifecycle observer at `DESTROYED`) and `rememberCoroutineScope()` (cancelled when the Composable leaves composition).

> **The takeaway:** none of this is magic. Every "advanced" coroutine feature is a small chunk of straightforward Kotlin built on top of three primitives — `Continuation`, `Job`, and `CoroutineDispatcher`. Once you internalize this, reading the kotlinx.coroutines source becomes routine instead of intimidating.

3. Coroutine builders

Builders create a coroutine from inside a scope. The four you'll use most:

BUILDER	RETURNS	USE WHEN	NOTES
`launch`	`Job`	Fire-and-forget side work	Exception cancels parent (unless supervisor)
`async`	`Deferred<T>`	You need a result; parallel decomposition	Exception is rethrown by `await()`
`runBlocking`	`T`	Tests, `main()`, bridging to blocking APIs	**Never** on Android main thread
`coroutineScope { }`	`T`	Inside a suspend function, run children + wait	Inherits caller's context; one fail cancels siblings

```
scope.launch { saveAnalytics(event) }                  // fire-and-forget

scope.launch {
    val a = async { api.user() }                       // parallel
    val b = async { api.orders() }
    render(a.await(), b.await())
}

suspend fun compose() = coroutineScope {               // inherit, wait for all
    val a = async { ... }
    val b = async { ... }
    a.await() to b.await()
}
```

> **Tip:** use `async` only when you actually fork into parallel work. Two back-to-back `async {...}.await()` calls is just sequential code with extra overhead.

4. CoroutineContext + Dispatchers

`CoroutineContext` is an indexed set of elements. Most important elements: `Job`, `CoroutineDispatcher`, `CoroutineName`, `CoroutineExceptionHandler`. Combine with `+`:

```
val ctx = Dispatchers.IO + CoroutineName("sync") + SupervisorJob() + handler
scope.launch(ctx) { ... }
```

A child coroutine's context = parent context + builder's argument context. The Job element is always replaced by the child's new Job (so structured concurrency works).

Dispatchers — what each is for

DISPATCHER	BACKED BY	USE FOR
`Main`	UI thread	UI updates, View/Compose state writes
`Main.immediate`	UI thread (skip dispatch if already there)	Same, slightly faster when already on Main
`Default`	Pool sized to `max(2, cpuCores)`	CPU-bound work: parsing, sorting, image processing
`IO`	Pool starting at 64, expands	Blocking I/O: Room, OkHttp synchronous, file reads
`Unconfined`	Caller thread initially, then resumed thread	Tests only — counter-intuitive in production
`limitedParallelism(N)`	Subset of Default/IO with N concurrent	Cap parallelism for resource-limited services

```
val dbDispatcher = Dispatchers.IO.limitedParallelism(4)        // don't flood DB
withContext(dbDispatcher) { dao.bulkInsert(rows) }
```

> **Tip:** `Default` and `IO` share the same underlying thread pool — switching between them is cheap and doesn't actually create a new thread.

What each dispatcher uses internally

DISPATCHER	BACKING	HOW IT DISPATCHES
`Main`	Platform main loop (Android `Looper`, Swing event queue, JavaFX app thread)	Posts a `Runnable` to the platform's main `Handler`/queue
`Main.immediate`	Same as `Main`	If already on the main thread, executes inline; else posts. Saves one dispatch hop
`Default`	`DefaultScheduler` — work-stealing pool sized to `max(2, cpuCores)`	Pushes onto a per-worker local deque; idle workers steal
`IO`	**Same DefaultScheduler**, gated by a semaphore (cap = `max(64, cpuCores)`)	Same threads as Default, but the semaphore lets the pool grow temporarily for blocking work
`Unconfined`	None	Runs in caller thread until first suspension; resumes wherever the awaited operation completed
`limitedParallelism(N)`	Wraps another dispatcher	Same threads, capped concurrency via internal semaphore

> **Key insight:** `Default` and `IO` are the *same threads*. The only difference is concurrency cap (cores vs 64+) and the assumption about whether work blocks. So `withContext(Dispatchers.IO) { ... }` from inside a Default coroutine doesn't allocate a new thread — it just changes the cap.

When to pick which (decision rules)

- **UI work (Compose recomposition, View mutation)** → `Main`; use `Main.immediate` in event handlers to skip the extra dispatch.
- **JSON parsing, sorting, image processing, crypto** → `Default`. CPU-bound; want N-core parallelism, not more.
- **Room queries, OkHttp synchronous calls, file I/O, JNI to a blocking C lib** → `IO`. The work mostly waits on the kernel; we can have many concurrent waiters.
- **Resource-limited services (slow DB, rate-limited HTTP)** → `Dispatchers.IO.limitedParallelism(N)`. Don't flood downstream.

- **Long-running blocking thread you fully own** → `Dispatchers.IO.limitedParallelism(1)` = your own private serialized worker.

5. Job hierarchy & structured concurrency

Every coroutine has a `Job`. Jobs form a tree mirroring the launch hierarchy. Five lifecycle states:

STATE	ISACTIVE	ISCOMPLETED	ISCANCELLED
New (lazy)	false	false	false
Active	true	false	false
Completing	true	false	false
Cancelling	false	false	true
Cancelled / Completed	false	true	(only Cancelled)

Rules of structured concurrency:

- Parent waits for all children to finish before completing.
- Cancelling the parent cancels all children.
- Failure of a child cancels the parent (and thus all siblings) — *unless* the parent is a `SupervisorJob` or you're inside `supervisorScope`.
- Cancellation propagates downward; failure propagates upward.

```
val parent = scope.launch {
    launch { childA() }
    launch { childB() }
}                           // parent completes only when A AND B finish

parent.cancel()             // cancels A and B too
parent.join()               // suspend until parent + children fully finished
```

Job vs SupervisorJob

JOB

One child failing cancels every sibling and the parent. Use when work is "all or nothing".

SUPERVISORJOB

One child failing leaves siblings alone. Use for independent parallel tasks (loading 3 widgets where one failing shouldn't kill the others).

How SupervisorJob actually differs internally

Both are `JobSupport` implementations. The only difference: `SupervisorJob` overrides `childCancelled(cause)` to return `false` — which means "I refuse to be cancelled because of my child's failure." A regular `Job` returns `true` there, which triggers cascading cancellation up to the root.

```
// Conceptually:
class JobImpl : JobSupport() {
    override fun childCancelled(cause: Throwable) = true     // propagate
}
class SupervisorJobImpl : JobSupport() {
    override fun childCancelled(cause: Throwable) = false    // don't propagate
}
```

When to use which

- **Use Job** for atomic operations where partial success is meaningless (e.g., parallel decomposition: load user + load orders, both must succeed).
- **Use SupervisorJob** at scope roots that host independent work (Application scope, ViewModelScope, screen-level scopes) so one failing job doesn't tear down the whole screen.
- **Don't mix them in the middle**: a SupervisorJob deep inside a Job tree only protects its direct children. Failures still propagate up *through* the SupervisorJob if it itself is cancelled.

Rule of thumb: `viewModelScope` / `lifecycleScope` / your custom app scopes use SupervisorJob by default. `coroutineScope { }` uses regular Job. `supervisorScope { }` is just `coroutineScope` with a SupervisorJob.

6. Cancellation

Cancellation throws `CancellationException` at the next suspension point. Tight CPU loops without suspension points are not cancellable until they call `yield()` or `ensureActive()`.

```
scope.launch {
    while (isActive) {                // or check ensureActive() periodically
        compute()
        yield()                      // suspend point — gives cancellation a chance
    }
}

// Bad: never cancellable
scope.launch { while (true) { computeForever() } }
```

Things to know:

- Standard library suspending functions (`delay`, `withContext`, `await`) check cancellation automatically.
- Custom `suspendCancellableCoroutine` wrappers should call `continuation.invokeOnCancellation { cleanup() }`.
- Cleanup that must run even after cancellation goes inside `withContext(NonCancellable) { ... }`. Use sparingly.
- Never catch `CancellationException` with a bare `catch (e: Throwable)` and swallow it — that breaks structured cancellation.

```
try { riskyCall() }
catch (e: CancellationException) { throw e }       // rethrow cancellation
catch (e: Throwable) { handle(e) }
```

7. Exception handling

Exceptions in coroutines behave differently between `launch` and `async`:

BUILDER	HOW EXCEPTIONS SURFACE
`launch`	Propagates immediately to parent Job; if not handled, goes to `CoroutineExceptionHandler` or default.
`async`	Stored in `Deferred`; rethrown only when you call `await()`. Also propagates to parent.

```
val handler = CoroutineExceptionHandler { _, e -> Log.e("App", "Uncaught", e) }
val scope = CoroutineScope(SupervisorJob() + Dispatchers.Main + handler)

scope.launch {
    try { api.call() }
    catch (e: IOException) { showError() }     // IO handled here
    // other exceptions propagate to handler
}

// async — must catch around await()
scope.launch {
    val d = async { riskyCall() }
    try { d.await() }
    catch (e: Exception) { handle(e) }
}
```

Where the handler actually runs

- **Only** attached to root Jobs (the Job at the top of a scope). Handlers on nested launches are ignored — exceptions bubble to the root.
- **Doesn't fire for** `async`: those exceptions surface via `await()`.
- If neither caught nor handler set: on JVM/Android, the exception goes to `Thread.UncaughtExceptionHandler` (Crashlytics catches it).

> **Pitfall:** A child throwing in a non-supervised scope cancels the entire scope — including unrelated parallel work. If that's not what you want, use `SupervisorJob` / `supervisorScope`.

8. Coroutine scope functions

"Scope functions" are suspend functions that create a new scope, execute a block, and wait for everything inside.

FUNCTION	WHAT IT DOES	FAILURE BEHAVIOR
`coroutineScope { }`	New scope; inherits caller's context (incl. dispatcher).	Any child fails → all cancelled, exception rethrown
`supervisorScope { }`	Same, but children failures don't cascade to siblings.	Each child handles its own failure
`withContext(ctx) { }`	Switch context (usually dispatcher) for one block.	Standard exception propagation
`withTimeout(ms) { }`	Cancel block if it exceeds time limit; throws `TimeoutCancellationException`.	Block is cancelled, exception thrown
`withTimeoutOrNull(ms) { }`	Same, but returns `null` on timeout instead of throwing.	Block cancelled, you get null

```
suspend fun loadAll(): Pair<User, List<Order>> = coroutineScope {
    val u = async { api.user() }
    val o = async { api.orders() }
    u.await() to o.await()
}

val result = withTimeoutOrNull(5_000) { api.fetch() } ?: defaultValue

val users = withContext(Dispatchers.IO) { dao.queryAll() }
```

9. Constructing your own CoroutineScope

A scope owns a Job and a default context. Three patterns:

```
// 1. App-wide singleton (custom)
val appScope = CoroutineScope(SupervisorJob() + Dispatchers.Default)

// 2. Lifecycle-scoped (Android Jetpack provides these)
viewModelScope.launch { ... }      // cleared in ViewModel.onCleared
lifecycleScope.launch { ... }      // cleared at Activity/Fragment DESTROYED

// 3. Per-task (don't usually do this — use scope functions instead)
val tempScope = CoroutineScope(Job())
tempScope.launch { ... }
tempScope.cancel()                  // you must cancel manually!
```

WHERE	USE
Inside a ViewModel	`viewModelScope`
Inside an Activity/Fragment	`lifecycleScope` + `repeatOnLifecycle`
Inside a Composable	`rememberCoroutineScope()`
Inside a long-lived Service / Application	Custom `CoroutineScope(SupervisorJob() + Dispatchers.X)`
Inside a suspending function	Don't create a scope — use `coroutineScope { }`

> **Anti-pattern:** `GlobalScope`. It's not cancellable, ignores structured concurrency, and leaks. Use a properly-scoped CoroutineScope tied to a lifecycle.

The full landscape — every scope, when each is right

SCOPE	JOB TYPE	DEFAULT DISPATCHER	CANCELLED WHEN	USE FOR
`GlobalScope`	None (top of tree)	`Default`	Never (manually)	Almost never. `@DelicateCoroutinesApi` for a reason
`MainScope()`	`SupervisorJob`	`Main`	`scope.cancel()`	Custom Activity/Fragment when not using Jetpack scopes
`CoroutineScope(...)`	Whatever you pass	Whatever you pass	`scope.cancel()`	Long-lived components (Application, Service) — must cancel manually
`viewModelScope`	`SupervisorJob`	`Main.immediate`	`onCleared()`	Inside ViewModel — default for VM work
`lifecycleScope`	`SupervisorJob`	`Main.immediate`	Lifecycle `DESTROYED`	Inside Activity/Fragment when not using Compose
`rememberCoroutineScope()`	Job linked to composition	`Main`	Composable leaves composition	Event handlers in Composables (clicks, drags)
`coroutineScope { }`	Inherits parent + child Job	Inherits	Block returns / failure	Inside a suspend function — preferred to creating a scope

GlobalScope — the deep dive (pros, cons, when it's actually OK)

What it actually is:

```
public object GlobalScope : CoroutineScope {
    override val coroutineContext: CoroutineContext = EmptyCoroutineContext
}
```

That's the entire definition. Empty context = no Job, no Dispatcher (defaults to `Dispatchers.Default` when launching). Coroutines launched here have **no parent** — they're roots of their own little trees, completely outside any structured-concurrency tree you might have.

PROS

- Survives any UI lifecycle — useful for fire-and-forget work that genuinely shouldn't be cancelled with the screen
- Zero ceremony — no setup, just launch
- Doesn't get cancelled when its launching code returns

CONS

- Bypasses structured concurrency (the whole point of coroutines)
- No automatic cancellation → easy to leak
- Tests can't control its scheduling without injecting a TestDispatcher
- Exceptions go to the default Thread.UncaughtExceptionHandler — surprising in tests
- `@DelicateCoroutinesApi` annotation in 1.5+ — IDE flags every usage
- Most "I need GlobalScope" turns out to be "I need a properly-scoped Application-level scope"

When it's acceptable (rare):

- Top-level analytics flush from `Application.onTerminate()` equivalent — never going to be cancelled anyway
- A library shipping fire-and-forget telemetry without a host lifecycle
- Your own deliberately-leaked daemon (you accept the leak)

When it's NOT acceptable (common):

- Network calls related to a screen → screen scope (`viewModelScope`)
- Background work that should survive process death → WorkManager, NOT GlobalScope
- "It works on my machine" code that leaks coroutines slowly until your test suite OOMs

Custom `CoroutineScope(...)` — the safer alternative to GlobalScope

```
class Session {
    private val scope = CoroutineScope(SupervisorJob() + Dispatchers.Default + CoroutineName("session"))

    fun start() { scope.launch { ... } }
    fun close() { scope.cancel() }       // CRITICAL — you must call this
}
```

Difference from GlobalScope:

- You own the Job — can `cancel()` as a unit when the session ends
- You can install a `CoroutineExceptionHandler` in the context
- Easy to swap dispatcher in tests
- Forgetting `cancel()` still leaks — but at least the leak is bounded to one scope

What's actually inside `viewModelScope`

```
// Conceptual implementation
val ViewModel.viewModelScope: CoroutineScope by lazy {
    val scope = CoroutineScope(SupervisorJob() + Dispatchers.Main.immediate)
    addCloseable(Closeable { scope.cancel() })   // auto-cancel on onCleared()
    scope
}
```

Why this combination:

- **SupervisorJob** — one failed coroutine doesn't tear down the entire ViewModel
- **Main.immediate** — most VM work updates UI; immediate skips a dispatch hop when already on Main
- **Closeable hook** — guaranteed cancel on `onCleared()`, no leaks

Decision tree (memorize this)

1. Inside a ViewModel? → `viewModelScope`
2. Inside a Composable event handler? → `rememberCoroutineScope()`
3. Inside an Activity/Fragment (View system)? → `lifecycleScope` + `repeatOnLifecycle(STARTED) { }`
4. Inside a CoroutineWorker.doWork? → just use `coroutineScope { }` inside; the Worker handles cancellation
5. Inside a long-lived custom component (Service, Application)? → `CoroutineScope(SupervisorJob() + dispatcher)` with manual `cancel()` on shutdown
6. Inside a suspend function? → **do NOT create a scope**; use `coroutineScope { }` or `supervisorScope { }`
7. Truly app-wide, no lifecycle, must never die? → custom Application-scoped `CoroutineScope` stored as a singleton — **not** GlobalScope

10. Shared mutable state — Mutex, Atomic, single-thread, copies

Coroutines run concurrently. If two coroutines race on a shared mutable variable, you get the same data races as with threads.

APPROACH	BEST FOR
`Mutex.withLock { }`	Coroutine-friendly mutual exclusion. Suspends instead of blocking.
`AtomicInteger / AtomicReference`	Simple counters / single-reference updates. CAS-based.
`limitedParallelism(1)` dispatcher	Single-threaded confinement; access state only through that dispatcher.
Immutable copies + StateFlow	Most modern Android code: emit a new state object instead of mutating.

```
// Mutex — coroutine-aware lock
val mutex = Mutex()
var counter = 0
launch { mutex.withLock { counter++ } }       // safe

// Single-thread dispatcher confinement
val stateDispatcher = Dispatchers.Default.limitedParallelism(1)
var state = State.Loading
suspend fun mutate() = withContext(stateDispatcher) { state = State.Loaded(...) }

// Modern preferred: immutable + StateFlow
val uiState = MutableStateFlow(UiState.Loading)
uiState.update { current -> current.copy(loading = false) }    // CAS under the hood
```

11. Channels

A `Channel` is a hot communication primitive — like a queue with suspending `send` / `receive`. Use for one-shot or persistent producer/consumer between coroutines.

CAPACITY	BEHAVIOR
`RENDEZVOUS` (default)	No buffer. `send` suspends until `receive`.
`BUFFERED` (default 64)	Buffered; `send` suspends only when full.
`UNLIMITED`	Never blocks `send`; memory unbounded.
`CONFLATED`	Buffer of 1; new `send` overwrites previous unread value.
Custom `capacity = N`	Fixed-size buffer.

```
val ch = Channel<Event>(Channel.BUFFERED)

// Producer
launch { events.forEach { ch.send(it) }; ch.close() }

// Consumer
launch { for (e in ch) handle(e) }    // loop ends when ch is closed

// Non-suspending send (for use in callbacks)
ch.trySend(event).isSuccess

// Fan-out: one producer, many consumers
repeat(4) { launch { for (e in ch) work(e) } }

// Fan-in: many producers, one consumer
launch { producerA(ch) }
launch { producerB(ch) }
launch { for (e in ch) consume(e) }
```

> **Tip:** for UI events with multiple subscribers, prefer `SharedFlow` over `Channel`. Channels are point-to-point.

12. `select` expression

Wait for the first of several suspending sources to complete. Like a multi-source `await`.

```
suspend fun raceFirst(a: Deferred<String>, b: Deferred<String>): String =
    select {
        a.onAwait { it }
        b.onAwait { it }
    }

// Receive from whichever Channel emits first
select<Unit> {
    chA.onReceive { handleA(it) }
    chB.onReceive { handleB(it) }
    onTimeout(1_000) { showTimeout() }
}
```

13. Hot vs cold sources

	COLD	HOT
Examples	Most `Flow` (`flow{}` , `flowOf` , `asFlow`)	`Channel` , `StateFlow` , `SharedFlow`
Producer starts when	A collector subscribes	Constructed (or first subscriber for `shareIn`)
Multiple collectors	Each gets a fresh execution	All share the same emissions
Late subscriber	Sees the whole flow from start	StateFlow: current value · SharedFlow: configurable replay · Channel: nothing
Termination	Completes when block ends	Lives until cancelled

14. Flow basics

A `Flow<T>` is a cold asynchronous sequence. The body inside `flow { }` only runs when something calls `collect` .

```
fun ticker(): Flow<Int> = flow {
    var i = 0
    while (true) { emit(i++); delay(1000) }
}

// Each collector restarts the producer
launch { ticker().collect { print("A:$it ") } }
launch { ticker().collect { print("B:$it ") } }
// Two independent timers
```

Three things define a Flow:

1. It's a description, not an execution. `val f = flow { emit(1) }` does nothing on its own.
2. Execution starts on the calling coroutine when `collect` is called.
3. Cancellation propagates: cancelling the collector's coroutine cancels the producer.

15. Flow building

BUILDER	USE
`flowOf(1, 2, 3)`	Static values.
`listOf(...).asFlow()`	Convert any `Iterable` / `Sequence` .
`flow { emit(...) }`	Custom emission logic; producer runs on the collector's coroutine.
`channelFlow { send(...) }`	Concurrent emission from inside a producer scope; allows multiple producer coroutines.
`callbackFlow { trySend(...); awaitClose { } }`	Wrap callback APIs (sensors, sockets, listeners).
`emptyFlow()` / `flow { }`	Empty stream.

```
// Wrapping a callback API into a Flow
fun locationUpdates(client: FusedLocationProviderClient): Flow<Location> = callbackFlow {
    val cb = object : LocationCallback() {
        override fun onLocationResult(r: LocationResult) {
            r.lastLocation?.let { trySend(it) }
        }
    }
    client.requestLocationUpdates(req, cb, Looper.getMainLooper())
    awaitClose { client.removeLocationUpdates(cb) }    // MUST unregister
}
```

Pitfall: `callbackFlow` won't compile without `awaitClose { }` — the framework forces you to specify cleanup.

16. Flow lifecycle hooks

Wrap a Flow with side effects at well-defined points:

```
flow
    .onStart      { showSpinner() }                  // before first emission
    .onEach       { Log.d("item", it.toString()) }   // each emission
    .onCompletion { cause -> hideSpinner() }         // success OR failure
    .catch        { e -> emit(fallback()) }          // upstream errors only
    .retry(3)    { e -> e is IOException }      // re-collect on certain errors
    .collect      { render(it) }
```

OPERATOR	NOTES
`onStart`	Can `emit` values; useful for placeholders.
`onEach`	Side-effect only; doesn't transform.
`onCompletion`	Receives the cause (null on success). Runs even on cancellation.
`catch`	Catches exceptions thrown *upstream*; transparent to downstream errors.
`retry / retryWhen`	Re-collect upstream from the start. Add backoff in `retryWhen`.

17. Flow operators — quick reference

Intermediate (return Flow)

OPERATOR	EFFECT
`map` / `mapNotNull`	1:1 transform / drop nulls.
`filter` / `filterNotNull` / `filterIsInstance`	Skip non-matching emissions.
`transform`	Emit zero, one, or many per input.
`scan` / `runningReduce`	Accumulate; emit each step.
`take(n)` / `drop(n)` / `takeWhile` / `dropWhile`	Slice the stream.
`distinctUntilChanged`	Skip consecutive equal emissions.
`debounce(ms)`	Emit only after silence of *ms*.
`sample(ms)`	Emit the most recent value every *ms*.
`flatMapConcat { }`	Sequential — wait for inner Flow to finish.
`flatMapMerge { }`	Concurrent — interleave emissions from inner Flows.
`flatMapLatest { }`	Cancel previous inner Flow on new outer value (search-style).
`combine(otherFlow) { }`	Emit on each change; uses latest of every source.
`zip(otherFlow) { }`	Pair-wise; waits for one from each.
`buffer(n)` / `conflate`	Decouple producer/consumer — see §19.
`flowOn(ctx)`	Switch the upstream context (dispatcher).

Terminal (suspend, return value)

OPERATOR	EFFECT
`collect { }`	Process each emission.
`collectLatest { }`	Cancel previous handler when a new value arrives.
`toList()` / `toSet()`	Materialize all emissions (must terminate).
`first()` / `firstOrNull()`	Cancel after first match.
`last()` / `single()`	Wait for completion.
`fold(init) { }`	Reduce to a single value.
`count()`	Count emissions.
`launchIn(scope)`	Launch collection in a scope; returns Job.

```
// Search-as-you-type pattern
queryFlow
    .debounce(300)
    .distinctUntilChanged()
    .flatMapLatest { q -> api.searchFlow(q) }
    .catch { emit(emptyList()) }
    .collect { render(it) }
```

18. SharedFlow & StateFlow

Hot replacements for LiveData. Use StateFlow for state, SharedFlow for events.

	STATEFLOW	SHAREDFLOW
Initial value	Required	Optional (replay)
Conflated	Yes (drops equal values)	No
Late subscriber sees	Current value	Configurable replay buffer
Best for	UI state (single source of truth)	One-shot events (toasts, navigation)
Thread safety	Yes	Yes

```
// State — current value of UI
val _state = MutableStateFlow(UiState.Loading)
val state: StateFlow<UiState> = _state.asStateFlow()
_state.value = UiState.Success(...)
// or atomic update
_state.update { it.copy(loading = false) }

// Events — one-shot, no conflation
val _events = MutableSharedFlow<Event>(replay = 0, extraBufferCapacity = 1)
val events: SharedFlow<Event> = _events.asSharedFlow()
viewModelScope.launch { _events.emit(Event.NavigateBack) }
```

Converting cold → hot: `stateIn` / `shareIn`

```
val users: StateFlow<List<User>> = repository.observeUsers()
    .stateIn(
        scope = viewModelScope,
        started = SharingStarted.WhileSubscribed(5_000),     // see below
        initialValue = emptyList()
    )
```

SHARINGSTARTED	WHEN UPSTREAM RUNS
`Eagerly`	Always (wastes battery if UI hidden)
`Lazily`	From first subscriber, never stops
`WhileSubscribed(stopTimeoutMillis = 5000)`	Starts on first sub, stops 5 s after last sub. **Recommended for UI Flows on Android.**

> **Why 5 seconds?** Bridges configuration changes (rotation typically < 1 s) and brief background trips, without keeping upstream alive when the user has truly navigated away.

19. Flow context, buffer, conflate

Flow respects context preservation: the producer runs on whatever dispatcher the collector is on, unless you use `flowOn`. `flowOn` only affects upstream operators.

```
flow {
    emit(loadFromDisk())            // runs on IO
}
.map { transform(it) }              // also on IO (still upstream of flowOn)
.flowOn(Dispatchers.IO)
.collect { render(it) }             // runs on caller's dispatcher (Main)
```

OPERATOR	EFFECT
`flowOn(ctx)`	Upstream runs on `ctx`; downstream stays on caller.
`buffer(n)`	Producer doesn't wait for consumer; intermediate queue of size *n*.
`conflate()`	If consumer is slow, drop intermediate values. Buffer size 1, "drop oldest".

```
fastProducer
    .conflate()                      // only the latest matters (e.g., scroll position)
    .collect { renderSlow(it) }

// or: process at producer's rate while consumer is slow
fastProducer
    .buffer(capacity = 10)
    .collect { processSlow(it) }
```

20. Testing coroutines

Use `runTest { }` from `kotlinx-coroutines-test`. It runs in virtual time — `delay(60_000)` completes immediately for the test.

```
@OptIn(ExperimentalCoroutinesApi::class)
class MyVmTest {
    private val dispatcher = StandardTestDispatcher()

    @Before fun setUp() { Dispatchers.setMain(dispatcher) }
    @After  fun tearDown() { Dispatchers.resetMain() }

    @Test fun loads_emits_success() = runTest {
        val repo = FakeRepo(User("42"))
        val vm = MyVm(repo)

        vm.state.test {                                // Turbine
            assertEquals(UiState.Loading, awaitItem())
            vm.load("42")
            advanceUntilIdle()                         // run pending coroutines
            assertEquals(UiState.Success(User("42")), awaitItem())
            cancelAndIgnoreRemainingEvents()
        }
    }
}
```

HELPER	WHAT IT DOES
`runTest { }`	Test scope with virtual time. Replaces `runBlocking`.
`StandardTestDispatcher`	Doesn't auto-run coroutines; you call `runCurrent` / `advanceTimeBy`.
`UnconfinedTestDispatcher`	Auto-runs each coroutine eagerly until first suspension. Easier but less precise.
`advanceTimeBy(ms)`	Move virtual time forward; runs scheduled coroutines.
`advanceUntilIdle()`	Run all scheduled coroutines until none left.
`runCurrent()`	Run currently scheduled (non-delayed) coroutines.
`flow.test { }` (Turbine)	Assert Flow emissions one by one with timeouts.

> **Tip:** use `StandardTestDispatcher` when timing matters (debounce, retry-with-delay). Use `UnconfinedTestDispatcher` for simple "did it emit?" tests.

21. Common patterns

Parallel decomposition

```
suspend fun loadAll() = coroutineScope {
    val u = async { api.user() }
    val o = async { api.orders() }
    val n = async { api.notifs() }
    Combined(u.await(), o.await(), n.await())
}
```

Sequential with shared timeout

```
withTimeout(5_000) {
    val a = stepOne()
    val b = stepTwo(a)
    stepThree(b)
}
```

Best-effort parallel (failure isolation)

```
supervisorScope {
    launch { runCatching { riskA() }.onFailure { Log.w(...) } }
    launch { runCatching { riskB() }.onFailure { Log.w(...) } }
}
```

Debounced search

```
queryFlow
    .debounce(300)
    .distinctUntilChanged()
    .flatMapLatest { q -> api.searchFlow(q) }
    .collect(::render)
```

Retry with exponential backoff

```
suspend fun <T> retry(times: Int = 3, base: Long = 200, block: suspend () -> T): T {
    var last: Throwable? = null
    repeat(times) { i ->
        try { return block() }
        catch (e: CancellationException) { throw e }
        catch (e: Throwable) {
            last = e
            delay(base * (1L shl i) + Random.nextLong(0, base))    // jitter
        }
    }
    throw last!!
}
```

Single-flight (dedupe concurrent calls for the same key)

```
private val inFlight = mutableMapOf<String, Deferred<User>>()
private val mu = Mutex()

suspend fun getUser(id: String): User {
    val existing = mu.withLock { inFlight[id] }
    if (existing != null) return existing.await()

    val deferred = scope.async { api.fetch(id) }
    mu.withLock { inFlight[id] = deferred }
    try { return deferred.await() }
    finally { mu.withLock { inFlight.remove(id) } }
}
```

22. Best practices — short list

- Use `suspend` functions in your repository / use-case layer; let them be cancelled by the caller's scope.
- Store mutable shared state behind a single source of truth (StateFlow + `update { }`), not scattered `var`s.
- Prefer `SharedFlow` + `StateFlow` over `Channel` for any UI-facing stream.
- For Android UI Flows, use `stateIn(viewModelScope, WhileSubscribed(5000), initial)` + `collectAsStateWithLifecycle()` in Compose.
- Always specify a dispatcher when entering blocking code (`withContext(Dispatchers.IO) { dao.query() }`).
- Use `SupervisorJob` for app-/screen-level scopes and for parallel branches that should be independent.
- In `callbackFlow`, never forget `awaitClose { unregister() }`.
- Pass test dispatchers via constructor (or via Hilt) so production scopes are testable.
- Prefer `runTest` + virtual time over `delay`-based real-time tests.
- Inject scopes into long-lived components (Application, Service) rather than using `GlobalScope`.
- Mark long-running suspend functions cancellable by inserting `yield()` or `ensureActive()` in CPU loops.
- Wrap exceptions at module boundaries — don't leak provider-specific types (HttpException, SQLiteException) up to the UI.

23. Common mistakes — anti-patterns

GLOBALSCOPE.LAUNCH

Bypasses structured concurrency, ignores cancellation, easy to leak. Use a properly-scoped CoroutineScope.

RUNBLOCKING ON MAIN

Blocks the UI thread; defeats the purpose of coroutines. Reserved for tests, `main()`, and bridging to legacy synchronous APIs.

CATCHING CANCELLATIONEXCEPTION

Swallowing it breaks structured cancellation. Always rethrow or catch a more specific subtype.

ASYNC WITHOUT AWAIT

Exception silently held by the Deferred. Either always await, or use `launch` with a handler.

TIGHT CPU LOOP WITHOUT YIELD

Not cancellable. Add `yield()` or `ensureActive()` periodically.

MUTABLE STATE ACROSS COROUTINES WITHOUT SYNC

Race conditions. Use `Mutex`, `Atomic`, or single-thread dispatcher confinement.

CALLING SUSPEND FROM INSIDE JAVA

You need a bridge: blocking (`runBlocking`), callback, `future { }`, or `promise { }`.

PLAIN `COLLECT` IN ACTIVITY/FRAGMENT

Continues collecting in background — drains battery. Use `repeatOnLifecycle(STARTED)` or `collectAsStateWithLifecycle`.

MULTIPLE PRODUCERS WRITING TO THE SAME STATEFLOW

Last write wins; intermediate states may be lost. Use a single owner or `update { }` for atomic compound updates.

FORGOT `AWAITCLOSE` IN CALLBACKFLOW

Compile error — but if you somehow trick it, you leak listeners.

24. Android integration cheat lines

```
// ViewModel
class MyVm(val repo: Repo) : ViewModel() {
    val state = repo.observe()
        .stateIn(viewModelScope, WhileSubscribed(5_000), UiState.Loading)
    fun refresh() = viewModelScope.launch { repo.refresh() }
}

// Compose
@Composable fun Screen(vm: MyVm = hiltViewModel()) {
    val state by vm.state.collectAsStateWithLifecycle()
    Render(state)
}

// Activity / Fragment (View system)
lifecycleScope.launch {
    repeatOnLifecycle(Lifecycle.State.STARTED) {
        vm.state.collect(::render)
    }
}

// Background work — survives process death
class SyncWorker(ctx: Context, p: WorkerParameters) : CoroutineWorker(ctx, p) {
    override suspend fun doWork(): Result {
        repo.sync()
        return Result.success()
    }
}

// Convert FCM-ish callback to Flow
fun incomingMessages(): Flow<Message> = callbackFlow {
    val listener = MessageListener { trySend(it) }
    api.subscribe(listener)
    awaitClose { api.unsubscribe(listener) }
}
```

★ Coroutines vs RxJava vs Executors vs Threads vs Callbacks

The five ways to do async work on the JVM/Android. Each has its place — but for new Kotlin/Android code in 2026, coroutines win for almost every use case. Here's the honest side-by-side.

	COROUTINES	RXJAVA	EXECUTORS / FUTURE	RAW THREADS	CALLBACKS
Primitive	`suspend fun` + `Continuation`	`Observable` + `Schedulers`	`Runnable/Callable` + `Future`	`Thread`	function references
Composition	Suspend funs compose like normal funs	Operator chains	Manual `Future.thenApply`	Manual	Nesting (callback hell)
Cancellation	Structured + automatic	`dispose()` — manual	`future.cancel()` — best effort	`thread.interrupt()` — best effort	Not built-in
Backpressure	Built-in (cold Flow); Channel for explicit	Built-in (Flowable + strategies)	Manual	Manual	Manual
Memory cost	~hundreds of bytes per coroutine	Subscription + closure overhead	~MB per Future + thread	~1 MB stack per thread	Just a closure
Concurrent count practical	Millions on a few threads	Thousands	Hundreds	Hundreds	n/a
Mental model	Sequential code	Reactive declarative pipelines	Producer/consumer	Imperative	Inverted control flow
Threading model	Cooperative on small pool (dispatchers)	Schedulers (io / computation / single)	OS-managed pool	OS-managed	Whoever called you
Library size	~20 KB	~1 MB	JDK builtin	JDK builtin	n/a
Test ergonomics	`runTest` + virtual time → great	`Schedulers.trampoline` + `TestObserver`	Hard	Hard	Hard
Lifecycle integration (Android)	First-class (`viewModelScope`, etc.)	External (`AutoDispose`, RxLifecycle)	None	None	Manual
Language support	`suspend` is a keyword	Library only	Library only	Library only	Language native (just functions)

Why coroutines won (the short version)

1. **Code reads sequentially.** Stack traces are real. Debugger steps line by line. No "everything is wrapped in a flatMap" mental gymnastics.
2. **Structured concurrency is the default.** No leaked work, no manual `CompositeDisposable` bookkeeping.
3. **Lighter than threads** (millions vs thousands), **lighter than RxJava runtime** (~50× smaller binary).
4. **Compiler-supported.** `suspend` is checked at compile time; you can only call suspend from suspend or a coroutine builder. RxJava can be misused without the compiler noticing.
5. **First-class Android lifecycle integration** via Jetpack — no third-party dance.

When to still pick something else (honest)

- **RxJava** — large existing codebase. Migrate *incrementally* using `kotlinx-coroutines-rx2/3` bridges (`asFlow()`, `rxSingle { }`); don't rewrite for rewrite's sake. Also: extremely complex reactive pipelines with 10+ chained operators where Rx's operator catalogue (~1000) is genuinely deeper than Flow's (~80).
- **Executors / CompletableFuture** — Java-only codebase, or you need to integrate with Java APIs that take an `Executor` (some frameworks do).
- **Raw Threads** — very rare. If you genuinely need a long-running blocking thread (e.g., custom event loop, native-API thread affinity), prefer `Dispatchers.IO.limitedParallelism(1)` or a `Service`. Don't reach for `Thread()` directly in Kotlin code.
- **Callbacks** — when wrapping platform APIs that *are* callbacks (CameraX, sensors, Bluetooth, View listeners). Wrap them into Flow/suspend at the boundary using `callbackFlow` or `suspendCancellableCoroutine`; everything above the boundary stays coroutines.

Conversion patterns (the bridges between worlds)

FROM → TO	CODE
Callback → suspend	`suspendCancellableCoroutine { cont -> api.fetch { cont.resume(it) } }`
Callback (multi-shot) → Flow	`callbackFlow { ...; awaitClose { unregister() } }`
Future → suspend	`future.await()` (from `kotlinx-coroutines-jdk8`)
RxJava Observable → Flow	`observable.asFlow()` (from `kotlinx-coroutines-rx2/3`)
RxJava Single → suspend	`single.await()`
Suspend → Future	`scope.future { suspendBlock() }`
Suspend → Callback	`scope.launch { val r = suspendBlock(); callback(r) }`
Blocking call → coroutine	`withContext(Dispatchers.IO) { blockingCall() }`
Thread → coroutine pool	Replace `Thread { ... }.start()` with `scope.launch(Dispatchers.IO) { ... }`
LiveData → Flow / vice versa	`liveData.asFlow()` / `flow.asLiveData()`

The decision tree for new code

1. **New Kotlin/Android code, any kind of async work?** → Coroutines.
2. **Wrapping a callback API?** → `callbackFlow` (multi-shot) or `suspendCancellableCoroutine` (one-shot). Above that boundary: coroutines.
3. **Working in an RxJava codebase?** → Use bridges. Don't migrate top-down; migrate edges first as you touch them.
4. **Pure Java / framework requires Executor?** → Use Executors. If your app code is Kotlin, wrap the Executor in a CoroutineDispatcher: `executor.asCoroutineDispatcher()`.
5. **Need a thread for native code or a long-running blocking loop?** → `Dispatchers.IO.limitedParallelism(1)`. Don't `new Thread()`.

25. One-page recall card

The 30-second mental model

Coroutines = suspendable computations. Compiler turns each `suspend` function into a state machine taking a `Continuation`. Suspension means returning `COROUTINE_SUSPENDED` with state preserved; resumption is a callback that jumps to the next state. Threads are not held while a coroutine waits.

Builders

- `launch` → fire-and-forget, returns `Job`
- `async` → result needed (parallel), returns `Deferred`
- `runBlocking` → tests / bridging only
- `coroutineScope { }` → all-or-nothing parallel
- `supervisorScope { }` → independent parallel
- `withContext(d) { }` → switch dispatcher for one block
- `withTimeout(ms) { }` / `withTimeoutOrNull`

Dispatchers

- **Main** — UI
- **Default** — CPU
- **IO** — blocking I/O
- **limitedParallelism(N)** — cap concurrency

Structured concurrency

- Parent waits for children to finish
- Cancel parent → cancels all children
- Child fails → cancels parent (unless supervisor)
- Cancellation = `CancellationException` at suspension points
- Cooperative — tight CPU loop needs `yield()` / `ensureActive()`

Hot vs cold

- Cold: `flow { }`, restarts per collector
- Hot: `StateFlow` (state + initial), `SharedFlow` (events + replay), `Channel` (point-to-point)
- `stateIn(WhileSubscribed(5000))` for UI flows

Flow operators you'll use daily

- `map`, `filter`, `distinctUntilChanged`
- `debounce`, `sample`, `scan`
- `flatMapLatest` (search), `flatMapMerge` (parallel), `flatMapConcat` (sequential)
- `combine` (latest of all), `zip` (pairwise)
- `flowOn` (upstream context), `buffer`, `conflate`
- `onStart` / `onEach` / `onCompletion` / `catch` / `retry`

Testing

- `runTest { }` + virtual time
- `StandardTestDispatcher` (control timing) vs `UnconfinedTestDispatcher` (eager)
- `advanceTimeBy`, `advanceUntilIdle`, `runCurrent`
- `Turbine.test { awaitItem() }` for Flow
- `Dispatchers.setMain(d)` in setUp; `resetMain()` in tearDown

Avoid

- `GlobalScope`
- `runBlocking` on Main
- Catching `CancellationException` with bare `Throwable`
- `async` without `await`
- Plain `collect` in lifecycle-aware code (use `repeatOnLifecycle`)
- Mutating shared state from multiple coroutines without sync
- Forgetting `awaitClose` in `callbackFlow`

Read this once for understanding · skim sections 22-25 the night before any Android interview. Pair with the **Top 210 Android Interview Questions** guide for full coverage.